



EXVUL

SMART CONTRACT **AUDIT REPORT**

Bric Smart Contract

DECEMBER 2025

Contents

1. EXECUTIVE SUMMARY	4
1.1 Methodology	4
2. FINDINGS OVERVIEW	7
2.1 Project Info And Contract Address	7
2.2 Summary	7
2.3 Key Findings	8
3. DETAILED DESCRIPTION OF FINDINGS	9
3.1 Missing read-only reentrancy protection allows oracle price manipulation during swap callbacks	9
3.2 Principal Trapping in Same Token Swaps	14
3.3 Token Metadata Collision via Dual-Prefix Stripping	16
3.4 Missing_scaleTrimInfo causes DoS in OKX Swaps	19
3.5 Arbitrary External Call to Theft	25
3.6 Permissionless market creation without reward token validation allows permanently unusable markets in registry	29
3.7 Missing _disableInitializers() in upgradeable implementation contract constructor . . .	32
3.8 Zero LP fee swaps via rounding error	34
3.9 Permanent ETH Locking Cause by payable	37
3.10 Typo in LimitOrderData Struct Field	40
4. CONCLUSION	41
5. APPENDIX	42
5.1 Basic Coding Assessment	42
5.1.1 Apply Verification Control	42
5.1.2 Authorization Access Control	42
5.1.3 Forged Transfer Vulnerability	42
5.1.4 Transaction Rollback Attack	43
5.1.5 Transaction Block Stuffing Attack	43
5.1.6 Soft Fail Attack Assessment	43
5.1.7 Hard Fail Attack Assessment	44
5.1.8 Abnormal Memo Assessment	44
5.1.9 Abnormal Resource Consumption	44
5.1.10 Random Number Security	45

5.2 Advanced Code Scrutiny	45
5.2.1 Cryptography Security	45
5.2.2 Account Permission Control	45
5.2.3 Malicious Code Behavior	46
5.2.4 Sensitive Information Disclosure	46
5.2.5 System API	46
6. DISCLAIMER	47
7. REFERENCES	48
8. About Exvul Security	49

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **Bric** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
Bric	17/12/2025 - 05/01/2026	Solidity

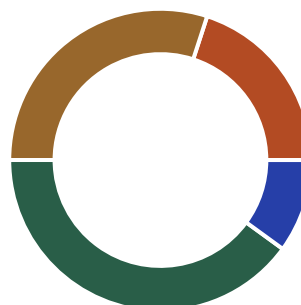
Repository

Commit Hash

045b3d5d074393ab3938fe35da843ad0

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	2
MEDIUM	3
LOW	4
INFO	1



2.3 Key Findings

Severity	Findings Title	Status
HIGH	Missing read-only reentrancy protection allows oracle price manipulation during swap callbacks	Fixed
HIGH	Principal Trapping in Same Token Swaps	Fixed
MEDIUM	Token Metadata Collision via Dual-Prefix Stripping	Fixed
MEDIUM	Missing_scaleTrimInfo causes DoS in OKX Swaps	Fixed
MEDIUM	Arbitrary External Call to Theft	Fixed
LOW	Permissionless market creation without reward token validation allows permanently unusable markets in registry	Fixed
LOW	Missing_disableInitializers() in upgradeable implementation contract constructor	Fixed
LOW	Zero LP fee swaps via rounding error	Fixed
LOW	Permanent ETH Locking Cause by payable	Fixed
INFO	Typo in LimitOrderData Struct Field	Fixed

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Missing read-only reentrancy protection allows oracle price manipulation during swap callbacks

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

```
contracts/core/Market/BricMarket.sol
contracts/oracles/lib/BricLpOracleLib.sol
contracts/core/erc20/BricERC20.sol
contracts/core/YieldContracts/BricPrincipalToken.sol
contracts/interfaces/IBMarket.sol
contracts/interfaces/IBPrincipalToken.sol
```

DESCRIPTION:

Oracle price manipulation during swap callbacks by exploiting the inconsistency between updated storage state and unverified token balances. `BricMarket.swapExactPtForSy()` and `swapSyForExactPt()` update state before the swap callback, creating a window where storage variables reflect post-swap values while actual token balances have not been verified yet.

```
// contracts/core/Market/BricMarket.sol
function swapExactPtForSy(
    address receiver,
    uint256 exactPtIn,
    bytes calldata data
) external nonReentrant notExpired returns (uint256 netSyOut, uint256
netSyFee) {
    MarketState memory market = readState(msg.sender);

    uint256 netSyToReserve;
    (netSyOut, netSyFee, netSyToReserve) =
market.swapExactPtForSy(YT.newIndex(), exactPtIn, block.timestamp);
```

```
        if (receiver != address(this)) IERC20(SY).safeTransfer(receiver,
netSyOut);
        IERC20(SY).safeTransfer(market.treasury, netSyToReserve);

        _writeState(market);    // Step 1: _storage.totalPt += exactPtIn,
        _storage.totalSy -= netSyOut

        if (data.length > 0) {

IBMarketSwapCallback(msg.sender).swapCallback(exactPtIn.neg(),
netSyOut.Int(), data);
        }    // Step 2: External callback, At this point _storage updated
        but PT balance not yet verified

        if (_selfBalance(PT) < market.totalPt.Uint())    // Step 3:
        Balance validation after callback
            revert Errors.MarketInsufficientPtReceived(_selfBalance(PT),
market.totalPt.Uint());

        emit Swap(msg.sender, receiver, exactPtIn.neg(), netSyOut.Int(),
netSyFee, netSyToReserve);
    }
// contracts/oracles/lib/BricLpOracleLib.sol
    function _getLpToAssetRateRaw(
        IBMarket market,
        uint32 duration,
        uint256 pyIndex
    ) private view returns (uint256 lpToAssetRateRaw) {
        // ...
        MarketState memory state = market.readState(address(0)); //
        Reads _storage.totalPt/totalSy directly
        // Price calculation based on updated storage but unverified
        balances
    }
// contracts/core/Market/BricMarket.sol
    function readState(address router) public view returns (MarketState
memory market) {
        market.totalPt = _storage.totalPt; // Returns post-swap value
        during callback
        market.totalSy = _storage.totalSy; // Returns post-swap value
        during callback
        market.totalLp = totalSupply().Int();
```

```
        // ...  
    }
```

Attack Vector:

```
contract Exploit is IBMarketSwapCallback {  
    function attack() external {  
        market.swapExactPtForSy(address(this), amount,  
            abi.encode("trigger"));  
    }  
  
    function swapCallback(int256, int256, bytes calldata) external {  
        // During this callback:  
        // - market._storage.totalPt increased  
        // - market._storage.totalSy decreased  
        // - But PT hasn't been transferred in yet  
  
        uint256 manipulatedPrice =  
            oracle.getLpToAssetRate(address(market), 1800);  
        // Price calculation uses inconsistent totalPt/totalSy ratio  
        // Can trigger improper liquidations in lending protocols  
        // Can enable over-borrowing against inflated collateral value  
    }  
}
```

IMPACT:

Attackers can manipulate LP oracle prices during swap callbacks by exploiting the inconsistency between updated storage state and unverified token balances, enabling over-borrowing or improper liquidations in lending protocols that rely on `BricPYLpOracle.getLpToAssetRate()` for collateral valuation.

RECOMMENDATIONS:

Add read-only reentrancy protection by exposing reentrancy guard state and adding oracle-side validation.

```
// contracts/core/erc20/BricERC20.sol  
contract BricERC20 is Context, IERC20, IERC20Metadata {  
    modifier nonReentrant() {
```

```

        require(_status != _ENTERED, "ReentrancyGuard: reentrant call");
        _status = _ENTERED;
        _;
        _status = _NOT_ENTERED;
    }
+
+   function reentrancyGuardEntered() external view virtual returns
+   (bool) {
+       return _reentrancyGuardEntered();
+   }
+
+   function _reentrancyGuardEntered() internal view returns (bool) {
+       return _status == _ENTERED;
+   }
+
// contracts/core/YieldContracts/BricPrincipalToken.sol
contract BricPrincipalToken is BricERC20, Initializable,
    IBPrincipalToken {
    function isExpired() public view returns (bool) {
        return MiniHelpers.isCurrentlyExpired(expiry);
    }
+
+   function reentrancyGuardEntered() external view override(BricERC20,
+   IBPrincipalToken) returns (bool) {
+       return _reentrancyGuardEntered();
+   }
+
}
// contracts/core/Market/BricMarket.sol
contract BricMarket is BricERC20, BricGauge, IBMarket {
    function isExpired() public view returns (bool) {
        return MiniHelpers.isCurrentlyExpired(expiry);
    }
+
+   function reentrancyGuardEntered() external view override(BricERC20,
+   IBMarket) returns (bool) {
+       return _reentrancyGuardEntered();
+   }
+
// contracts/oracles/lib/BricLpOracleLib.sol
library BricLpOracleLib {
    function _getLpToAssetRateRaw(
        IBMarket market,
        uint32 duration,
        uint256 pyIndex
    ) private view returns (uint256 lpToAssetRateRaw) {

```

```
+      _checkMarketReentrancy(market);
      MarketState memory state = market.readState(address(0));
      // ...
    }

+
+      function _checkMarketReentrancy(IBMarket market) internal view {
+          try market.reentrancyGuardEntered() returns (bool entered) {
+              require(entered == false, "ReentrancyGuard: reentrant call");
+          } catch {
+              return;
+          }
+      }
+  }
// contracts/interfaces/IBMarket.sol
interface IBMarket {
    function redeemRewards(address user) external returns (uint256[]
memory);
+
+      function reentrancyGuardEntered() external view returns (bool);
// contracts/interfaces/IBPrincipalToken.sol
interface IBPrincipalToken {
    function isExpired() external view returns (bool);
+
+      function reentrancyGuardEntered() external view returns (bool);
}
```

3.2 Principal Trapping in Same Token Swaps

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**`contracts/router/swap-aggregator/base/SwapAggregatorBase.sol`**DESCRIPTION:**

`_swapExactInput` incorrectly calculates the output amount `netTokenOut` when `tokenIn` is the same as `tokenOut` during arbitrage or loop transactions (e.g., USDC -> ETH -> USDC) will trap the user's principal amount.

```
// contracts/router/swap-aggregator/base/SwapAggregatorBase.sol
function _swapExactInput(SwapDataInfo calldata swapDataInfo, address
sender) internal returns (uint256 amountOut) {
    // ...
    uint256 preBalance = _selfBalance(swapDataInfo.tokenOut);
    _swap(swapDataInfo.tokenIn, swapDataInfo.amountIn,
swapDataInfo.swapData);
    uint256 netTokenOut = _selfBalance(swapDataInfo.tokenOut) -
preBalance;
    // ...
}
```

IMPACT:

When `tokenIn == tokenOut`, the `preBalance` check is performed after the user's principal amountIn has already been transferred into the contract via `_transferIn` in the calling function. Therefore: `preBalance = InitialBalance + Principal`. After the swap loop, the balance is roughly `InitialBalance + Principal + Profit`. The calculation `netTokenOut = CurrentBalance - preBalance` results in `netTokenOut = Profit`. The contract then transfers only this `netTokenOut` profit back to the user, effectively trapping the user's principal amountIn within the contract.

RECOMMENDATIONS:

Adjust the `preBalance` calculation to account for the incoming principal when `tokenIn == tokenOut`.

```
// contracts/router/swap-aggregator/base/SwapAggregatorBase.sol

function _swapExactInput(SwapDataInfo calldata swapDataInfo, address
sender) internal returns (uint256 amountOut) {
    // approve token to external router
    _approveForExtRouter(swapDataInfo.tokenIn, swapDataInfo.swapData);

    uint256 preBalance = _selfBalance(swapDataInfo.tokenOut);
+   if (swapDataInfo.tokenIn == swapDataInfo.tokenOut) {
+       preBalance -= swapDataInfo.amountIn;
+   }

    _swap(swapDataInfo.tokenIn, swapDataInfo.amountIn,
swapDataInfo.swapData);
    uint256 netTokenOut = _selfBalance(swapDataInfo.tokenOut) -
preBalance;
```

3.3 Token Metadata Collision via Dual-Prefix Stripping

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**`contracts/core/YieldContracts/BricYieldContractFactory.sol``contracts/core/libraries/StringLib.sol`**DESCRIPTION:**

`_stripSYPrefix()` applies both `SY_NAME_PREF` ("SY") and `SY_SYMBOL_PREF` ("SY-") stripping operations sequentially to both name and symbol fields.

```
// contracts/core/YieldContracts/BricYieldContractFactory.sol
function _stripSYPrefix(string memory _str) internal pure returns
(string memory) {
    StringLib.slice memory str = _str.toSlice();
    StringLib.slice memory delim_name = SY_NAME_PREF.toSlice();    //
    "SY "
    StringLib.slice memory delim_symbol = SY_SYMBOL_PREF.toSlice();
    // "SY-"
    return str.beyond(delim_name).beyond(delim_symbol).toString();
    // tries to remove both prefixes
}

function createYieldContract(
    address SY,
    uint32 expiry,
    bool doCacheIndexSameBlock
) external returns (address PT, address YT) {
    // ...
    string memory syCoreName = _stripSYPrefix(_SY.name());
    string memory syCoreSymbol = _stripSYPrefix(_SY.symbol());
    // ...
}
```

`beyond()` removes a prefix if matched, otherwise returns the string unmodified. By chaining two `beyond()` calls, it attempts to remove both "SY" and "SY-" from the same string, regardless of whether

it is a name or symbol.

Scenario:

A malicious actor can deploy a custom SY token with intentionally swapped prefixes.

Malicious SY Token

name: "SY-USDC"

symbol: "SY USDC"

Legitimate SY: name="SY USDC", symbol="SY-USDC"

-> name.beyond("SY").beyond("SY-") = "USDC" -> symbol.beyond("SY").beyond("SY-") = "USDC" -> PT
Symbol: "PT-USDC-31Dec2025"

Malicious SY: name="SY-USDC", symbol="SY USDC"

-> name.beyond("SY") = "SY-USDC" .beyond("SY-") = "USDC"

-> symbol.beyond("SY") = "USDC"

.beyond("SY-") = "USDC"

-> PT Symbol: "PT-USDC-31Dec2025" //Identical to legitimate token

IMPACT:

Attackers can generate PT/YT tokens with metadata identical to legitimate high value tokens (USDC, USDT, DAI, etc.) by deploying malicious SY wrappers around worthless assets. Users relying on metadata in UIs cannot distinguish malicious tokens from authentic ones.

RECOMMENDATIONS:

Handle the prefix stripping differently for name and symbol.

```
// contracts/core/libraries/StringLib.sol
library StringLib {
    // ...
+   function stripPrefix(string memory _str, string memory _prefix)
+       internal pure returns (string memory) {
+       slice memory s = toSlice(_str);
+       slice memory d = toSlice(_prefix);
+       return toString(beyond(s, d));
+   }
}
// contracts/core/YieldContracts/BricYieldContractFactory.sol
```

```
function createYieldContract(
    address SY,
    uint32 expiry,
    bool doCacheIndexSameBlock
) external returns (address PT, address YT) {
    // ...
    IStandardizedYield _SY = IStandardizedYield(SY);
    (, , uint8 assetDecimals) = _SY.assetInfo();

-     string memory syCoreName = _stripSYPrefix(_SY.name());
-     string memory syCoreSymbol = _stripSYPrefix(_SY.symbol());
+     string memory syCoreName = _SY.name().stripPrefix(SY_NAME_PREF);
+     string memory syCoreSymbol =
        _SY.symbol().stripPrefix(SY_SYMBOL_PREF);

    // ...
}

- function _stripSYPrefix(string memory _str) internal pure returns
  (string memory) {
-     StringLib.slice memory str = _str.toSlice();
-     StringLib.slice memory delim_name = SY_NAME_PREF.toSlice();
-     StringLib.slice memory delim_symbol = SY_SYMBOL_PREF.toSlice();
-     return str.beyond(delim_name).beyond(delim_symbol).toString();
- }
```

3.4 Missing_scaleTrimInfo causes DoS in OKX Swaps

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**`contracts/router/swap-aggregator/OKXScaleHelper.sol`**DESCRIPTION:**

When BricSwapAggregator performs an OKX swap that requires scaling `actualAmount < rawAmount`, the standard parameters like `minReturn` in function arguments are scaled correctly. But any validation parameters encoded in the calldata tail handled by `_scaleTrimInfo` remain unscaled.

```
function _okxScaling(
    bytes calldata rawCallData,
    uint256 actualAmount
) internal pure returns (bytes memory scaledCallData) {
    // ...
    } else if (selector == IOKXDexRouter.smartSwapById.selector)
    {
        (
            uint256 orderId,
            IOKXDexRouter.BaseRequest memory baseRequest,
            uint256[] memory batchesAmount,
            IOKXDexRouter.RouterPath[][] memory batches,
            IOKXDexRouter.PMMSwapRequest[] memory extraData
        ) = abi.decode(
            dataToDecode,
            (
                uint256,
                IOKXDexRouter.BaseRequest,
                uint256[],
                IOKXDexRouter.RouterPath[][],
                IOKXDexRouter.PMMSwapRequest[]
            )
        );

        batchesAmount = _scaleArray(batchesAmount, actualAmount,
            baseRequest.fromTokenAmount);
```

```

        baseRequest.minReturnAmount = (baseRequest.minReturnAmount *
actualAmount) / baseRequest.fromTokenAmount;
        baseRequest.fromTokenAmount = actualAmount;

        return abi.encodeWithSelector(selector, orderId, baseRequest,
batchesAmount, batches, extraData);
    } else {
        revert("BricSwap: OKX selector not supported");
    }
}

```

IMPACT:

In the OKX Router receiving a discrepancy: - Input: Scaled Down (e.g., 90% of original). - Tail Expectation: Original 100% value. OKX Router will likely revert the transaction because the execution result based on 90% input will fail to meet the unscaled (100%) expectation encoded in the extra data.

RECOMMENDATIONS:

Add calldata tail scaling logic to `_okxScaling`.

```
// contracts/router/swap-aggregator/OKXScaleHelper.sol

+ uint256 internal constant TRIM_FLAG_MASK =
0xffffffffffffffff00000000000000000000000000000000000000000000000000000;
+ uint256 internal constant TRIM_FLAG =
0x7777777711110000000000000000000000000000000000000000000000000000;
+ uint256 internal constant TRIM_DUAL_FLAG =
0x77777777222200000000000000000000000000000000000000000000000000;
+ uint256 internal constant TRIM_EXPECT_AMOUNT_OUT_MASK =
0x000000000000000000000000fffffffffffaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa;

function _okxScaling(
    bytes calldata rawCallData,
    uint256 actualAmount
) internal pure returns (bytes memory scaledCallData) {
    bytes4 selector = bytes4(rawCallData[:4]);
    bytes calldata dataToDecode = rawCallData[4:];
+     uint256 rawAmountIn;

    if(selector == IOKXDexRouter.uniswapV3SwapTo.selector) {
```

```

        (uint256 receiver, uint256 amount, uint256 minReturn,
uint256[] memory pools) = abi.decode(
            dataToDecode,
            (uint256, uint256, uint256, uint256[]))
    );
+   rawAmountIn = amount;
    minReturn = (minReturn * actualAmount) / amount;
    amount = actualAmount;

-   return abi.encodeWithSelector(selector, receiver, amount,
minReturn, pools);
+   scaledCallData = abi.encodeWithSelector(selector, receiver,
amount, minReturn, pools);
    } else if (selector == IOKXDexRouter.smartSwapTo.selector) {
        (
            uint256 orderId,
            address receiver,
            IOKXDexRouter.BaseRequest memory baseRequest,
            uint256[] memory batchesAmount,
            IOKXDexRouter.RouterPath[][] memory batches,
            IOKXDexRouter.PMMSwapRequest[] memory extraData
        ) = abi.decode(
            dataToDecode,
            (
                uint256,
                address,
                IOKXDexRouter.BaseRequest,
                uint256[],
                IOKXDexRouter.RouterPath[][] ,
                IOKXDexRouter.PMMSwapRequest[]
            )
        );
+   rawAmountIn = baseRequest.fromTokenAmount;
    batchesAmount = _scaleArray(batchesAmount, actualAmount,
baseRequest.fromTokenAmount);
    baseRequest.minReturnAmount = (baseRequest.minReturnAmount *
actualAmount) / baseRequest.fromTokenAmount;
    baseRequest.fromTokenAmount = actualAmount;

-   return abi.encodeWithSelector(selector, orderId, receiver,
baseRequest, batchesAmount, batches, extraData);
+   scaledCallData = abi.encodeWithSelector(selector, orderId,
receiver, baseRequest, batchesAmount, batches, extraData);

```

```

    } else if (selector == IOKXDexRouter.unxswapTo.selector) {
        (uint256 srcToken, uint256 amount, uint256 minReturn, address
receiver, bytes32[] memory pools) = abi
            .decode(dataToDecode, (uint256, uint256, uint256,
address, bytes32[]));
+       rawAmountIn = amount;
        minReturn = (minReturn * actualAmount) / amount;
        amount = actualAmount;

-       return abi.encodeWithSelector(selector, srcToken, amount,
minReturn, receiver, pools);
+       scaledCallData = abi.encodeWithSelector(selector, srcToken,
amount, minReturn, receiver, pools);
    } else if (selector == IOKXDexRouter.unxswapByOrderId.selector) {
        (uint256 srcToken, uint256 amount, uint256 minReturn,
bytes32[] memory pools) = abi.decode(
            dataToDecode,
            (uint256, uint256, uint256, bytes32[])
        );
+       rawAmountIn = amount;
        minReturn = (minReturn * actualAmount) / amount;
        amount = actualAmount;

-       return abi.encodeWithSelector(selector, srcToken, amount,
minReturn, pools);
+       scaledCallData = abi.encodeWithSelector(selector, srcToken,
amount, minReturn, pools);
    } else if (selector == IOKXDexRouter.smartSwapByOrderId.selector)
{
    (
        uint256 orderId,
        IOKXDexRouter.BaseRequest memory baseRequest,
        uint256[] memory batchesAmount,
        IOKXDexRouter.RouterPath[][] memory batches,
        IOKXDexRouter.PMMSwapRequest[] memory extraData
    ) = abi.decode(
        dataToDecode,
        (
            uint256,
            IOKXDexRouter.BaseRequest,
            uint256[],
            IOKXDexRouter.RouterPath[][],
            IOKXDexRouter.PMMSwapRequest[]

```

```

        )
    );
+     rawAmountIn = baseRequest.fromTokenAmount;
    batchesAmount = _scaleArray(batchesAmount, actualAmount,
baseRequest.fromTokenAmount);
    baseRequest.minReturnAmount = (baseRequest.minReturnAmount *
actualAmount) / baseRequest.fromTokenAmount;
    baseRequest.fromTokenAmount = actualAmount;

-     return abi.encodeWithSelector(selector, orderId, baseRequest,
batchesAmount, batches, extraData);
+     scaledCallData = abi.encodeWithSelector(selector, orderId,
baseRequest, batchesAmount, batches, extraData);
    } else {
        revert("BricSwap: OKX selector not supported");
    }
+     scaledCallData = _scaleTrimInfo(rawCallData, scaledCallData,
actualAmount, rawAmountIn);
+     return scaledCallData;
    }

+     function _scaleTrimInfo(
+         bytes calldata rawCallData,
+         bytes memory scaledCallData,
+         uint256 actualAmountIn,
+         uint256 rawAmountIn
+     ) internal pure returns (bytes memory) {
+         bytes memory tail = rawCallData[scaledCallData.length:];
+         // search trim flag from back to front, get the index
+         assembly {
+             let endPtr := add(add(tail, 32), mload(tail))
+             let startPtr := add(tail, 32)
+             let shown := 0
+
+             for { let i := sub(endPtr, 32) } or(gt(i, startPtr), eq(i,
startPtr)) { i := sub(i, 32) } {
+                 let data := mload(i)
+                 if or(eq(and(data, TRIM_FLAG_MASK), TRIM_FLAG),
eq(and(data, TRIM_FLAG_MASK), TRIM_DUAL_FLAG)) {
+                     shown := add(shown, 1)
+                     if eq(shown, 2) {
+                         let expectAmountOut := and(data,
TRIM_EXPECT_AMOUNT_OUT_MASK)

```

```
+             let acutalExpectAmountOut := mul(expectAmountOut,
+ actualAmountIn)
+             acutalExpectAmountOut :=
+ div(acutalExpectAmountOut, rawAmountIn)
+             acutalExpectAmountOut :=
+ and(acutalExpectAmountOut, TRIM_EXPECT_AMOUNT_OUT_MASK)
+             data := and(data,
+ not(TRIM_EXPECT_AMOUNT_OUT_MASK))
+             data := or(data, acutalExpectAmountOut)
+             mstore(i, data)
+             break
+         }
+     }
+ }
+ scaledCallData = bytes.concat(scaledCallData, tail);
+ return scaledCallData;
+ }
```


3.5 Arbitrary External Call to Theft

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**`contracts/router/swap-aggregator/BricSwapAggregator.sol``contracts/router/swap-aggregator/base/SwapAggregatorBase.sol`**DESCRIPTION:**

swapExactInput allows users to specify an arbitrary extRouter address and extCalldata. SwapAggregatorBase indefinitely approves this user-controlled extRouter and executes the call without any validation or whitelisting.

```
// contracts/router/swap-aggregator/BricSwapAggregator.sol
function swapExactInput(
    SwapDataInfo calldata swapDataInfo
) external payable nonReentrant returns (uint256 amountOut) {
    address sender = msg.sender;

    require(swapDataInfo.amountIn > 0, "BricSwap: Invalid amountIn");

    _transferIn(swapDataInfo.tokenIn, sender, swapDataInfo.amountIn);

    amountOut = _swapExactInput(swapDataInfo, sender);
}

// contracts/router/swap-aggregator/base/SwapAggregatorBase.sol
function _swapExactInput(SwapDataInfo calldata swapDataInfo, address
sender) internal returns (uint256 amountOut) {
    // approve token to external router
    _approveForExtRouter(swapDataInfo.tokenIn, swapDataInfo.swapData);

    uint256 preBalance = _selfBalance(swapDataInfo.tokenOut);
    _swap(swapDataInfo.tokenIn, swapDataInfo.amountIn,
swapDataInfo.swapData);
    uint256 netTokenOut = _selfBalance(swapDataInfo.tokenOut) -
preBalance;

    if (netTokenOut < swapDataInfo.minOut) {
```

```
        revert InsufficientAmountOut(netTokenOut,
swapDataInfo.minOut);
    }

    _transferOut(swapDataInfo.tokenOut, swapDataInfo.receiver,
netTokenOut);

    emit SwapExactEvent(
        swapDataInfo.swapData.swapType,
        sender,
        swapDataInfo.tokenIn,
        swapDataInfo.amountIn,
        swapDataInfo.receiver,
        swapDataInfo.tokenOut,
        netTokenOut
    );

    return netTokenOut;
}

function _approveForExtRouter(address token, SwapData calldata data)
internal {
    if (token == NATIVE) return;

    if (data.swapType == SwapType.OKX) {
        _safeApproveInfV2(IERC20(token), _okx_getTokenApprove());
    } else {
        _safeApproveInfV2(IERC20(token), data.extRouter);
    }
}
}
```

IMPACT:

Attacker can supply a malicious contract like extRouter, receive infinite approval for any token held by the contract, and drain all funds by calling transferFrom.

RECOMMENDATIONS:

Implement an allowlist for external routers.

```
// contracts/router/swap-aggregator/base/SwapAggregatorBase.sol
```

```
import "../OKXScaleHelper.sol";
import "../IBSwapAggregator.sol";
import "../ParaswapScaleHelper.sol";
+import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

abstract contract SwapAggregatorBase is
    IBSwapAggregator,
    TokenHelper,
    OKXScaleHelper,
    ParaswapScaleHelper,
+   Ownable,
    ReentrancyGuard
{
    using Address for address;
    using SafeERC20 for IERC20;

    address private constant KYBER_SCALING_HELPER =
    0x2f577A41BeC1BE1152AeEA12e73b7391d15f655D;
+
+   mapping(address => bool) public allowedRouters;
+   event SetAllowedRouter(address indexed router, bool allowed);

+   function setAllowedRouter(address router, bool allowed) external
    onlyOwner {
+       allowedRouters[router] = allowed;
+       emit SetAllowedRouter(router, allowed);
+   }
+
    // BricRouter Call
    function swap(address tokenIn, uint256 amountIn, SwapData calldata
    data) external payable nonReentrant {
        _approveForExtRouter(tokenIn, data);

        _swap(tokenIn, amountIn, data);

        emit SwapSingle(data.swapType, tokenIn, amountIn);
    }

    // ...

    function _approveForExtRouter(address token, SwapData calldata data)
```

```
internal {
    if (token == NATIVE) return;

    if (data.swapType == SwapType.OKX) {
        _safeApproveInfV2(IERC20(token), _okx_getTokenApprove());
    } else {
+       require(allowedRouters[data.extRouter], "BricSwap: Router
not allowed");
        _safeApproveInfV2(IERC20(token), data.extRouter);
    }
}

function _safeApproveInfV2(IERC20 token, address spender) internal {
    if (token.allowance(address(this), spender) < type(uint256).max)
{
        token.forceApprove(spender, type(uint256).max);
    }
}

function _swap(address tokenIn, uint256 amountIn, SwapData calldata
data) internal {
+       require(allowedRouters[data.extRouter], "BricSwap: Router not
allowed");
    data.extRouter.functionCallWithValue(
        data.needScale ? _getScaledInputData(data.swapType,
data.extCalldata, amountIn) : data.extCalldata,
        tokenIn == NATIVE ? amountIn : 0
    );
}
```

3.6 Permissionless market creation without reward token validation allows permanently unusable markets in registry

SEVERITY:

LOW

STATUS:

Fixed

PATH:

contracts/core/Market/BricMarketFactory.sol

contracts/core/Market/BricGauge.sol

DESCRIPTION:

The factory performs PT validation but skips SY checks. Since `BricGauge._getRewardTokens()` enforces a `MAX_REWARD_TOKENS = 20` limit at runtime, any SY returning more than 20 reward tokens will cause all LP token operations (mint/burn/transfer) to revert with `TooManyRewardTokens` error. However, the market deployment and registration complete successfully before any runtime check occurs.

```
// contracts/core/Market/BricMarketFactory.sol
function createNewMarket(
    address PT,
    int256 scalarRoot,
    int256 initialAnchor,
    uint80 lnFeeRateRoot
) external returns (address market) {
    if (!IBYieldContractFactory(yieldContractFactory).isPT(PT))
        revert Errors.MarketFactoryInvalidPt();
    if (IBPrincipalToken(PT).isExpired()) revert
        Errors.MarketFactoryExpiredPt();
    if (lnFeeRateRoot > maxLnFeeRateRoot)
        revert
            Errors.MarketFactoryLnFeeRateRootTooHigh(lnFeeRateRoot,
                maxLnFeeRateRoot);

    if (markets[PT][scalarRoot][initialAnchor][lnFeeRateRoot] !=
        address(0))
        revert Errors.MarketFactoryMarketExists();

    if (initialAnchor < minInitialAnchor)
```

```
        revert Errors.MarketFactoryInitialAnchorTooLow(initialAnchor,
minInitialAnchor);

        market = BaseSplitCodeFactory._create2(
            0,
            bytes32(block.chainid),
            abi.encode(PT, scalarRoot, initialAnchor, lnFeeRateRoot /*,
veBric, gaugeController */),
            marketCreationCodeContractA,
            marketCreationCodeSizeA,
            marketCreationCodeContractB,
            marketCreationCodeSizeB
        );

        markets[PT][scalarRoot][initialAnchor][lnFeeRateRoot] = market;

        if (!allMarkets.add(market)) assert(false); // @audit Permanently
        registered, no removal mechanism

        emit CreateNewMarket(market, PT, scalarRoot, initialAnchor,
lnFeeRateRoot);
    }
```

```
// contracts/core/Market/BricGauge.sol
uint256 private constant MAX_REWARD_TOKENS = 20;

function _getRewardTokens() internal view virtual override returns
(address[] memory) {
    address[] memory SYRewards =
IStandardizedYield(SY).getRewardTokens();
    // if (SYRewards.contains(BRIC)) return SYRewards;
    // return SYRewards.append(BRIC);
    if (SYRewards.length > MAX_REWARD_TOKENS) {
        revert Errors.TooManyRewardTokens(SYRewards.length,
MAX_REWARD_TOKENS);
    }
    return SYRewards;
}
```

IMPACT:

When `SY.getRewardTokens().length > 20`, any LP token transfer/mint/burn triggers `_beforeTokenTransfer()` -> `_updateAndDistributeRewardsForTwo()` -> `_getRewardTokens()`, causing immediate revert. The market exists in registry but is completely unusable.

RECOMMENDATIONS:

Implement SY reward token validation before market deployment.

```
// contracts/core/Market/BricMarketFactory.sol
+import {IStandardizedYield} from "
+    ../../interfaces/IStandardizedYield.sol";

+    uint256 private constant MAX_REWARD_TOKENS = 20;

    function createNewMarket(
        address PT,
        int256 scalarRoot,
        int256 initialAnchor,
        uint80 lnFeeRateRoot
    ) external returns (address market) {
        if (!IBYieldContractFactory(yieldContractFactory).isPT(PT))
            revert Errors.MarketFactoryInvalidPt();
        if (IBPrincipalToken(PT).isExpired()) revert
            Errors.MarketFactoryExpiredPt();
        if (lnFeeRateRoot > maxLnFeeRateRoot)
            revert
                Errors.MarketFactoryLnFeeRateRootTooHigh(lnFeeRateRoot,
                    maxLnFeeRateRoot);
+
+    address SY = IBPrincipalToken(PT).SY();
+    address[] memory rewardTokens =
        IStandardizedYield(SY).getRewardTokens();
+    if (rewardTokens.length > MAX_REWARD_TOKENS) {
+        revert Errors.TooManyRewardTokens(rewardTokens.length,
            MAX_REWARD_TOKENS);
+    }

        if (markets[PT][scalarRoot][initialAnchor][lnFeeRateRoot] !=
            address(0))
            revert Errors.MarketFactoryMarketExists();
```

3.7 Missing `_disableInitializers()` in upgradeable implementation contract constructor

SEVERITY:

LOW

STATUS:

Fixed

PATH:`contracts/core/YieldContracts/BricYieldContractFactory.sol`**DESCRIPTION:**

BricYieldContractFactory inherits from BoringOwnableUpgradeable (which extends OpenZeppelin's Initializable) but fails to call `_disableInitializers()` in constructor. This violates OpenZeppelin's security best practice for upgradeable contracts.

```
// contracts/core/YieldContracts/BricYieldContractFactory.sol
contract BricYieldContractFactory is BoringOwnableUpgradeable,
    IBYieldContractFactory {
    // ...

    constructor(
        address _ytCreationCodeContractA,
        uint256 _ytCreationCodeSizeA,
        address _ytCreationCodeContractB,
        uint256 _ytCreationCodeSizeB
    ) {
        ytCreationCodeContractA = _ytCreationCodeContractA;
        ytCreationCodeSizeA = _ytCreationCodeSizeA;
        ytCreationCodeContractB = _ytCreationCodeContractB;
        ytCreationCodeSizeB = _ytCreationCodeSizeB;
    }

    function initialize(
        uint96 _expiryDivisor,
        uint128 _interestFeeRate,
        uint128 _rewardFeeRate,
        address _treasury
    ) external initializer {
        __BoringOwnable_init(msg.sender);
        setExpiryDivisor(_expiryDivisor);
        setInterestFeeRate(_interestFeeRate);
    }
}
```



```
        setRewardFeeRate(_rewardFeeRate);  
        setTreasury(_treasury);  
    }  
}
```

IMPACT:

Without disabling initializers on the implementation contract, an attacker could initialize it directly, potentially taking ownership or misconfiguring implementation state in upgradeable deployments.

RECOMMENDATIONS:

Add `_disableInitializers()` call in the constructor to prevent direct initialization of the implementation contract.

```
// contracts/core/YieldContracts/BricYieldContractFactory.sol  
    constructor(  
        address _ytCreationCodeContractA,  
        uint256 _ytCreationCodeSizeA,  
        address _ytCreationCodeContractB,  
        uint256 _ytCreationCodeSizeB  
    ) {  
        ytCreationCodeContractA = _ytCreationCodeContractA;  
        ytCreationCodeSizeA = _ytCreationCodeSizeA;  
        ytCreationCodeContractB = _ytCreationCodeContractB;  
        ytCreationCodeSizeB = _ytCreationCodeSizeB;  
+        _disableInitializers();  
    }
```

3.8 Zero LP fee swaps via rounding error

SEVERITY:

LOW

STATUS:

Fixed

PATH:

contracts/core/Market/BricMarket.sol

DESCRIPTION:

BricMarket.swapExactPtForSy() and swapSyForExactPt() calculates the netSyFee (fee paid to LPs). However, it fails to verify that this fee is no zero in terms of the underlying asset. If the fee is no zero in SY terms but rounds down to zero in Asset terms (due to exchange rate or precision), LPs receive effectively zero value.

```
// contracts/core/Market/BricMarket.sol
function swapExactPtForSy(
    address receiver,
    uint256 exactPtIn,
    bytes calldata data
) external nonReentrant notExpired returns (uint256 netSyOut, uint256
netSyFee) {
    MarketState memory market = readState(msg.sender);

    uint256 netSyToReserve;
    (netSyOut, netSyFee, netSyToReserve) =
market.swapExactPtForSy(YT.newIndex(), exactPtIn, block.timestamp);

    if (receiver != address(this)) IERC20(SY).safeTransfer(receiver,
netSyOut);
    IERC20(SY).safeTransfer(market.treasury, netSyToReserve);

    _writeState(market);

    if (data.length > 0) {

IBMarketSwapCallback(msg.sender).swapCallback(exactPtIn.neg(),
netSyOut.Int(), data);
    }
```

```
        if (_selfBalance(PT) < market.totalPt.Uint())
            revert Errors.MarketInsufficientPtReceived(_selfBalance(PT),
market.totalPt.Uint());

        //@audit missing asset fee check

        emit Swap(msg.sender, receiver, exactPtIn.neg(), netSyOut.Int(),
netSyFee, netSyToReserve);
    }
```

IMPACT:

Dust swaps or swaps with specific low amounts where netSyFee is positive but syToAsset(netSyFee) is zero. By repeatedly executing such swaps, users can trade without paying effective fees to LPs.

RECOMMENDATIONS:

Add the missing check to ensure LPs receive no zero value.

```
// contracts/core/Market/BricMarket.sol

function swapExactPtForSy(
    address receiver,
    uint256 exactPtIn,
    bytes calldata data
) external nonReentrant notExpired returns (uint256 netSyOut, uint256
netSyFee) {
    MarketState memory market = readState(msg.sender);
+   PYIndex index = YT.newIndex();

    uint256 netSyToReserve;
-   (netSyOut, netSyFee, netSyToReserve) =
market.swapExactPtForSy(YT.newIndex(), exactPtIn, block.timestamp);
+   (netSyOut, netSyFee, netSyToReserve) =
market.swapExactPtForSy(index, exactPtIn, block.timestamp);

    if (receiver != address(this)) IERC20(SY).safeTransfer(receiver,
netSyOut);
    IERC20(SY).safeTransfer(market.treasury, netSyToReserve);

    _writeState(market);
}
```

```
        if (data.length > 0) {

IPMarketSwapCallback(msg.sender).swapCallback(exactPtIn.neg(),
netSyOut.Int(), data);
        }

        if (_selfBalance(PT) < market.totalPt.Uint()) {
            revert Errors.MarketInsufficientPtReceived(_selfBalance(PT),
market.totalPt.Uint());
        }

+         if (index.syToAsset(netSyFee - netSyToReserve) == 0) {
+             revert Errors.MarketZeroNetLPFee();
+         }

        emit Swap(msg.sender, receiver, exactPtIn.neg(), netSyOut.Int(),
netSyFee, netSyToReserve);
    }
```

3.9 Permanent ETH Locking Cause by payable

SEVERITY:

LOW

STATUS:

Fixed

PATH:`contracts/router/swap-aggregator/BricSwapAggregator.sol``contracts/core/libraries/TokenHelper.sol`**DESCRIPTION:**

swapExactInput, swapExactInputWithPermit, and swapExactInputWithPermit2 are payable functions, allowing users to send ETH. When swapping ERC20 tokens (tokenIn != NATIVE), msg.value is ignored by the _transferIn logic in TokenHelper.

```
// contracts/router/swap-aggregator/BricSwapAggregator.sol
function swapExactInput(
    SwapDataInfo calldata swapDataInfo
) external payable nonReentrant returns (uint256 amountOut) {
    address sender = msg.sender;

    require(swapDataInfo.amountIn > 0, "BricSwap: Invalid amountIn");

    _transferIn(swapDataInfo.tokenIn, sender, swapDataInfo.amountIn);

    amountOut = _swapExactInput(swapDataInfo, sender);
}
// contracts/core/libraries/TokenHelper.sol
function _transferIn(address token, address from, uint256 amount)
internal {
    if (token == NATIVE) require(msg.value == amount, "eth mismatch");
    else if (amount != 0) IERC20(token).safeTransferFrom(from,
address(this), amount);
}
```

IMPACT:

Users who accidentally attach ETH to an ERC20 swap transaction will lose that ETH. The funds are

permanently locked in the contract as there is no sweep or rescue function.

RECOMMENDATIONS:

Revert the transaction if `msg.value > 0` when `tokenIn` is not the native token.

```
// contracts/router/swap-aggregator/BricSwapAggregator.sol
function swapExactInput(
    SwapDataInfo calldata swapDataInfo
) external payable nonReentrant returns (uint256 amountOut) {
    address sender = msg.sender;

    require(swapDataInfo.amountIn > 0, "BricSwap: Invalid amountIn");
+   if (swapDataInfo.tokenIn != address(0)) {
+       require(msg.value == 0, "BricSwap: Do not send ETH");
+   }

    _transferIn(swapDataInfo.tokenIn, sender, swapDataInfo.amountIn);

    amountOut = _swapExactInput(swapDataInfo, sender);
}

function swapExactInputWithPermit(
    SwapDataInfo calldata swapDataInfo,
    uint256 deadline,
    bytes memory signature
) public payable nonReentrant returns (uint256 amountOut) {
    address sender = msg.sender;

+   if (swapDataInfo.tokenIn != address(0)) {
+       require(msg.value == 0, "BricSwap: Do not send ETH");
+   }
+

    _checkPermit(swapDataInfo.tokenIn, swapDataInfo.amountIn,
        deadline, signature);
    // ...
    _transferIn(swapDataInfo.tokenIn, sender, swapDataInfo.amountIn);

    amountOut = _swapExactInput(swapDataInfo, sender);
}

function swapExactInputWithPermit2(
    SwapDataInfo calldata swapDataInfo,
```

```
        uint256 nonce,
        uint256 deadline,
        bytes calldata signature
    ) public payable nonReentrant returns (uint256 amountOut) {
        address sender = msg.sender;

+         if (swapDataInfo.tokenIn != address(0)) {
+             require(msg.value == 0, "BricSwap: Do not send ETH");
+         }
+
        _checkPermit(swapDataInfo.tokenIn, swapDataInfo.amountIn,
            deadline, signature);
        // ...
        amountOut = _swapExactInput(swapDataInfo, sender);
    }
```

3.10 Typo in LimitOrderData Struct Field

SEVERITY:

INFO

STATUS:

Fixed

PATH:`contracts/interfaces/IBallActionType.sol``contracts/router/base/ActionBase.sol`**DESCRIPTION:**

LimitOrderData struct field epsSkipMarket has been wrongly renamed to epsSkIBMarket.

```
// contracts/interfaces/IBallActionType.sol
struct LimitOrderData {
    address limitRouter;
    uint256 epsSkIBMarket; // "Skip" -> "Sk" + "IB"
    FillOrderParams[] normalFills;
    FillOrderParams[] flashFills;
    bytes optData;
}
```

IMPACT:

This typo can lead to integration mistakes or misunderstandings in downstream contracts and tooling that rely on the struct field name for clarity.

RECOMMENDATIONS:

Rename the field to epsSkipMarket for clarity.

```
// contracts/interfaces/IBallActionType.sol
struct LimitOrderData {
    address limitRouter;
-   uint256 epsSkIBMarket;
+   uint256 epsSkipMarket;
    // ...
}
```



```
// contracts/router/base/ActionBase.sol
function _fillLimit(
    address receiver,
    IERC20 tokenIn,
    uint256 netInput,
    LimitOrderData calldata lim
) internal returns (uint256 netLeft, uint256 netOut, uint256 netSyFee,
    bool doMarketOrder) {
    // ...
-   doMarketOrder = netLeft > netInput.mulDown(lim.epsSkIBMarket);
+   doMarketOrder = netLeft > netInput.mulDown(lim.epsSkipMarket);
}
```

4. CONCLUSION

In this audit, we thoroughly analyzed **Bric** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-684: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cwe.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

Contact

 **Website**
www.exvul.com

 **Twitter**
[@EXVULSEC](https://twitter.com/EXVULSEC)

 **Email**
contact@exvul.com

 **Github**
github.com/EXVUL-Sec